

MODULE 3 · CONFIGURATION

Secrets

Sensitive data, decoupled from images — but not encrypted by default

Credentials don't belong in images or ConfigMaps

Passwords, tokens and certs get hardcoded into images or dumped into ConfigMaps far too often. A **Secret** is a dedicated API object for this data — kept separate from application config, and mounted into Pods without ever baking it into an image.

But "Secret" is a name, not a guarantee. By default the values are only **base64-encoded** — that's the gotcha this lab is really about.

BY THE END YOU CAN

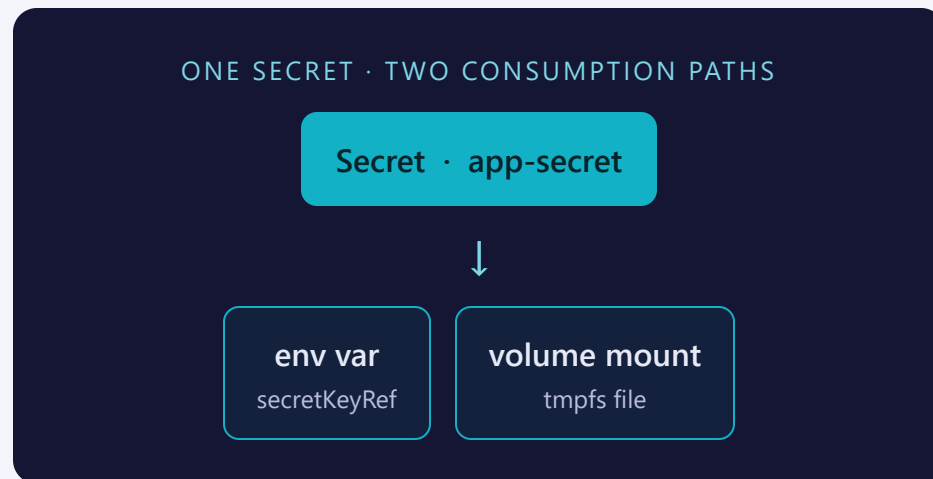
- Create a Secret two ways
- Explain data vs stringData
- Consume a Secret as env vars and as a volume
- Explain why base64 ≠ encryption

CONCEPT

A Secret is a ConfigMap shaped for sensitive values

- Same shape as a ConfigMap — a namespaced key-value bundle a Pod can reference
- Values are stored **base64-encoded**, not plaintext, in the object
- Mounted Secrets live in an in-memory tmpfs, never written to node disk
- A `type` field marks its shape so tooling knows what to expect

The API server treats it as sensitive by convention — that alone doesn't make it secret.



One object, several built-in shapes

TYPE	USED FOR
<code>Opaque</code>	Generic key-value data — the default, most common type
<code>kubernetes.io/tls</code>	TLS certificate + key (<code>tls.crt</code> , <code>tls.key</code>)
<code>kubernetes.io/dockerconfigjson</code>	Registry credentials for pulling private images
<code>kubernetes.io/basic-auth</code>	<code>username</code> + <code>password</code> fields
<code>kubernetes.io/service-account-token</code>	Auto-issued API credentials (Lab 7.3 — ServiceAccounts)

MENTAL MODEL

The type field is a label some tooling checks — Kubernetes doesn't validate or protect the contents any harder because of it.

data is base64, stringData is plaintext-in

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: training
type: Opaque
data:
  username: YWRtaW4=
  password: UzNjcjN0UEBzc3dvcnQ=
```

- **data** — values must already be base64-encoded; it's what's actually stored, and what get/describe return
- Could write stringData instead with plaintext values — the API server base64-encodes them into data on write, but stringData itself never round-trips back out
- namespace — Secrets are namespaced, same as ConfigMaps
- type: Opaque — the default shape for generic key-value data

Imperative encodes for you, declarative you own

IMPERATIVE — FAST, ONE-OFF

```
kubectl create secret generic app-secret --from-literal=username=admin
```

Base64-encodes the value for you automatically

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f secret.yaml
```

You own the file — but you must base64-encode `data` values yourself

THE BRIDGE TRICK

Generate the YAML (already base64-encoded) instead of encoding by hand:

```
$ kubectl create secret generic app-secret \  
  --from-literal=username=admin --dry-run=client -o yaml > secret.yaml
```

SEE IT

Secret as environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret-env
  namespace: training
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - echo Username=$DB_USER Password=$DB_PASS && sleep 3600
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: username
        - name: DB_PASS
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: password
      restartPolicy: Never
```

SEE IT

Secret as a mounted volume

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-secret-volume
  namespace: training
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - cat /etc/secrets/username && echo " " && cat /etc/secrets/password && sleep 3600
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-secret
  restartPolicy: Never
```


SEE IT

Env var or mounted file — same Secret, different exposure

Env vars are visible to `kubectl exec -- env`, process listings and crash dumps, and are fixed at container start.

A mounted volume is delivered as read-only files under `/etc/secrets`, and the kubelet syncs updated values to disk without a Pod restart.

PREFER

Volume mounts over env vars — env vars are easy to leak into logs and dumps that outlive the Pod.

THE PUNCHLINE

Base64 is encoding, not encryption

Anyone with read access to the Secret can reverse it in one command — there's no key, no ciphertext, just a text transform:

```
$ kubectl get secret app-secret -o jsonpath='{.data.password}' \
| base64 -d
```

By default that same base64 string is what sits in **etcd** too — unencrypted.

WHAT ACTUALLY PROTECTS A SECRET

- Encryption at rest for etcd
- RBAC — restrict get/list on secrets
- Least-privilege ServiceAccounts per Pod

Where people trip

- **Treating base64 as protection.** It's reversible with zero secrets of its own — it's just text-safe binary.
- **Committing secret.yaml to git.** Base64 in version control is a leaked credential in version control.
- **Same key in data and stringData.** stringData silently wins.
- **Expecting env vars to auto-update.** Only mounted volumes pick up a Secret change without a Pod restart.

Commands to keep at hand

```
$ kubectl create secret generic NAME --from-literal=k=v    # imperative, auto base64
$ kubectl get secret NAME -o jsonpath='{.data.KEY}' \
  | base64 -d                                             # decode one value
$ kubectl describe secret NAME                           # keys + byte counts, no values
$ kubectl create secret tls NAME --cert=c.crt --key=k.key # TLS pair
$ kubectl get secret NAME -o yaml                        # full object, base64 data
```

Tip: describe never shows values — reach for get -o yaml plus base64 -d to check content.

RECAP

Secrets in one breath

- Secret = ConfigMap shape for sensitive values, not encrypted by default
- type picks the shape — Opaque, tls, dockerconfigjson, ...
- data is base64 already; stringData is plaintext-in, encoded on write
- Consume as env (simple, can leak) or volume (files, safer, live-updates)
- Real protection = encryption at rest + RBAC, not base64

HANDS-ON

Now run Lab 3.2
[labs/3.2/](#)

Next: [4.1 — Services](#)